



Especificação do Projeto: Pedra-Papel-Tesoura Fase 3 (Sistemas Distribuídos e Sockets)

Objetivo do Trabalho: Nas fases anteriores, vocês construíram uma base sólida. Na Fase 1, modelaram as regras de negócio complexas do Pedra-Papel-Tesoura (apostas, All-In, desistência) utilizando boas práticas de POO. Na Fase 2, o desafio foi a concorrência: transformar os jogadores em Threads independentes e dominar a sincronização para evitar condições de corrida (*race conditions*).

Agora, na Fase 3, daremos o passo definitivo para o mundo real: vamos quebrar a aplicação em partes físicas distintas. O objetivo é transformar o jogo em um Sistema Distribuído Cliente-Servidor, comunicando-se através de rede utilizando Sockets TCP.

Vocês não terão mais instâncias locais conversando na mesma memória. O Servidor rodará em um processo (ou máquina), e os Clientes rodarão em outros [Para simular, poderá executar todos no localhost].

1. Arquitetura do Sistema

O sistema deverá ser refatorado para operar no seguinte modelo:

- Servidor (ServerSocket): Será o coordenador. Ele abre uma porta de rede, escuta conexões de entrada, emparelha os jogadores em partidas e gerencia os saldos. Cada cliente conectado deve ser tratado por uma Thread separada (reaproveitando a implementação da Fase 2).
- Cliente (Socket): A interface do jogador. Conecta-se ao servidor, envia objetos com suas ações (Apostar, Jogar, Desistir) e aguarda as respostas e resultados.
- Protocolo de Comunicação: A troca de dados será feita através da Serialização Nativa do Java. Vocês trafegarão instâncias de classes (como Aposto, Jogada, ou Mensagem) usando ObjectOutputStream e ObjectInputStream.

2. O Desafio Principal: Tolerância a Falhas

Em um ambiente distribuído, a rede é imprevisível. O cliente não é confiável: ele pode ter uma queda de energia, perder o sinal do Wi-Fi ou simplesmente demorar uma eternidade para decidir sua jogada. **O Servidor nunca pode travar ou bloquear a partida indefinidamente por culpa de um cliente problemático.**

Seu sistema deverá obrigatoriamente implementar tratamento para dois cenários de falha:

1. **Regra do W.O. por Tempo (Timeout)**: O servidor deve estipular um tempo máximo (ex: 30 segundos) para aguardar a ação de um jogador. Se o tempo for excedido sem resposta, o jogador inativo perde por W.O. A partida é encerrada, os fundos apostados retornam ou são repassados ao adversário (definam a regra) e os recursos são liberados.
2. **Desconexão Abrupta (Crash)**: Se um cliente "puxar o cabo de rede" ou fechar o terminal abruptamente no meio de uma rodada, o servidor deve detectar a queda da conexão instantaneamente, cancelar a rodada em andamento sem corromper os saldos e notificar o jogador sobrevivente de que o adversário abandonou o jogo.

3. Dicas de Ouro para o Desenvolvimento

Junto com este documento, estou enviando alguns arquivos de exemplo (ServidorJogo.java, ClienteJogo.java, TrataJogador.java e MensagemJogo.java) que demonstram a base da conexão, o envio de um objeto e a captura das exceções de rede.

Ao integrar essa base ao jogo completo de vocês, prestem muita atenção nestes três pontos cruciais:

- **A "Armadilha" do Buffer e a Importância do flush():** Fluxos de rede em Java utilizam *buffers* (espaços de memória temporária) por eficiência. Se você fizer apenas `out.writeObject(minhaJogada);`, o Java pode guardar a mensagem no buffer e não enviá-la para a rede, esperando para agrupar mais dados. Isso fará o outro lado ficar travado eternamente esperando. **Sempre que quiser que o dado viaje pela rede imediatamente, chame `out.flush()`; logo após a escrita.**
- **A Ordem de Criação dos Fluxos (Streams):** Sempre instancie o `ObjectOutputStream` e faça um `flush()` antes de instanciar o `ObjectInputStream`. O construtor do fluxo de entrada em Java tenta ler um cabeçalho da rede imediatamente. Se ambos os lados (Cliente e Servidor) tentarem criar a leitura primeiro, os dois ficarão bloqueados esperando, gerando um *deadlock* de conexão.
- **A Interface Serializable:** Qualquer classe que vá viajar pelo Socket precisa ter implements `Serializable`. É altamente recomendado adicionar o atributo `private static final long serialVersionUID = 1L;` nessas classes. Isso evita erros de incompatibilidade caso a classe seja levemente modificada no futuro.

4. Avaliação

O critério principal desta etapa é a **resiliência**. Durante a avaliação, os testes incluirão:

- Derrubar propositalmente jogador no meio de um All-In. O servidor deve lidar com a exceção (ex: `EOFException` ou `SocketException`), preservar o estado do sistema e liberar o adversário.
- Deixar um cliente inativo para forçar a `SocketTimeoutException`.
- Verificar se não há envio de *Strings* puras e inseguras; toda a lógica deve continuar orientada a objetos transitando pelo fluxo.
- Sempre, no termino do jogo, faça uma auditoria na quantidade de fichas (elas não podem desaparecer!).

5. Observações

A observação abaixo depende muito da forma que implementaram a Fase 2:

- Na versão que tenho, não precisei alterar nada nas classes: Jogo, Mesa, FaseRodada e Jogada. Acabei criando/editando as classes: MensagemJogo, PedraPapelTesoura (contém main do servidor, que pergunta a quantidade de jogadores e aguarda a conexão de todos para iniciar), Jogador, ClienteBot (contém main do cliente).

Bom trabalho e explorem os exemplos enviados para entender a mecânica dos fluxos antes de plugar toda a lógica do Pedra-Papel-Tesoura!